



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХТ

Лекция 18. Управление программными интерфейсами приложений (API)

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Программные интерфейсы приложений
-  Создание открытых экосистем ПО
-  RESTful API и средства их описания
-  Спецификация OpenAPI

Что такое API?

Application Programming Interface

- 📡 программный интерфейс приложения, или интерфейс прикладного программирования, или прикладной интерфейс программирования
- 📡 набор готовых классов, процедур, функций, структур и констант, предоставляемых приложением (библиотекой, сервисом) или операционной системой для использования во внешних программных продуктах (Википедия)
- 📡 определяет функциональность, которую предоставляет приложение (библиотека, сервис), при этом позволяет абстрагироваться от того, как именно эта функциональность реализована



Почему API так важны сегодня?

Новый взгляд на API

- ◉ API – самостоятельный публичный продукт
- ◉ API – канал сбыта для бизнеса

Количество публичных API постоянно растет

- ◉ По данным ProgrammableWeb, в 2017 году было доступно более 17000 публичных API, в 2005 году – единицы

Объем интернет трафика, приходящийся на вызовы API, для многих интернет платформ превышает трафик к веб интерфейсам

API – основа для построения динамичных цифровых экосистем

Новые термины

- ◉ API First
- ◉ API Value Chain
- ◉ API Economy

Новый взгляд на API

Публичные API

- ◉ Доступны через Интернет, сведения о них публикуются в Интернет

API – это продукт

- ◉ Предназначен для удовлетворения конкретных потребностей потребителей
- ◉ Меняется вместе с потребностями потребителя
- ◉ Объект маркетинга

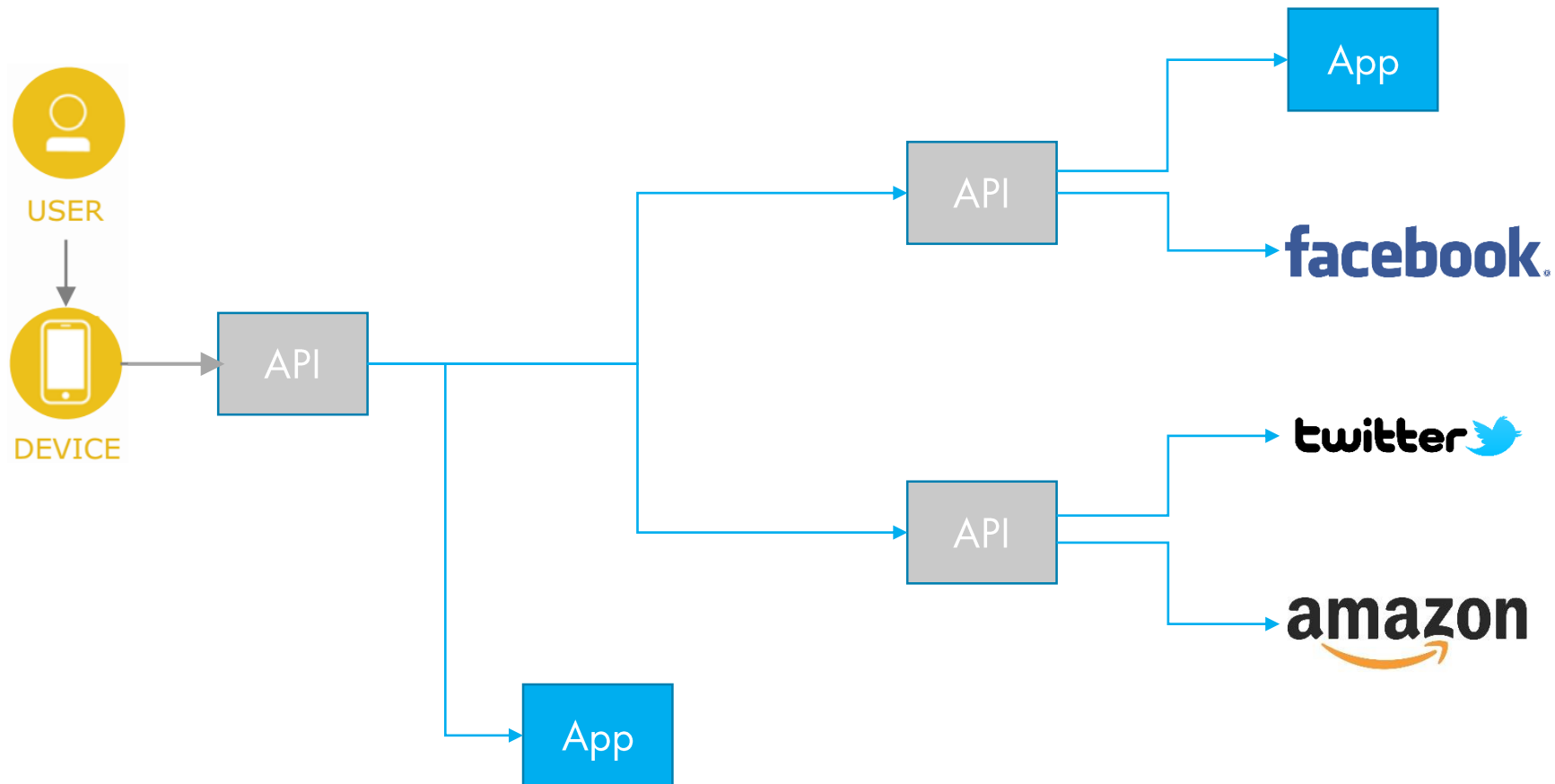
API – канал сбыта, часть бизнеса предприятия

API могут быть непосредственно монетизированы

Оппортунистский подход

- ◉ API создаются ради попытки использовать новую возможность на рынке
- ◉ API быстро меняются
- ◉ API не рассчитаны на повторное использование

Цепочки API



Новый взгляд на API и SOA

- Service Oriented Architecture – многократное использование
 - Службы создаются с расчетом на многократное использование
 - Интерфейсы проектируются так, чтобы оставаться неизменными длительное время
 - Развитие инфраструктуры ведется вокруг внутренних или открытых стандартов
- API – оппортунизм
 - API можно рассматривать как службу, но созданную для конкретного потребителя
 - Максимально отражает требования потребителя
 - Меняется вместе с требованиями потребителя
- API могут играть роль фасада для служб
 - SOA изолирует потребителей от изменений в реализации служб
 - API изолирует службы от изменений требований потребителя

Реализация API

Технологии реализации интерфейса API

- ⦿ REST – описание URI ресурсов и операций
- ⦿ SOAP – описание методов и структур данных
- ⦿ MQTT – описание событий и сообщений

Требования к платформе реализации API

- ⦿ Высокая скорость разработки и изменений при минимальных затратах
- ⦿ Высокопроизводительная среда выполнения
- ⦿ Надежная модель безопасности
- ⦿ Поддержка политик, контролирующих использование API
- ⦿ Управление жизненным циклом API
- ⦿ Публикация API в публичный доступ
- ⦿ Аналитика использования API
- ⦿ Монетизация

Целостный подход к управлению API



HTTP API

- ☞ Интеграция на основе API широко используется в современных системах
 - ☉ Абстрактные API эффективно изолирует клиента от службы
 - Изменения реализации службы, не затрагивают клиентов
 - ☉ Легкоизменяемые API создают представления службы, отвечающие текущим ожиданиям клиентов
- ☞ Публикация API требует механизма формального описания
- ☞ RESTful API – широко используемая на практике реализация
- ☞ Для описания RESTful API используют:
 - ☉ RAML (RESTful API Modeling Language)
 - Использует синтаксис YAML
 - ☉ OpenAPI
 - См. следующий слайд

OpenAPI

Наследник проекта Swagger API

- ⦿ Создан SmartBear Software в 2010 году

С 2015 года развивается под управлением Linux Foundation

- ⦿ Поддерживается Google, IBM, Microsoft и др.

Использует синтаксис JSON или YAML

Актуальная версия OpenAPI — 3.1.0

- ⦿ Полностью совместима с JSON Schema



Сравнение языков разметки

```
<!-- XML -->
<elements>
  <element>
    <symbol>H</symbol>
    <number>1</number>
    <weight>1</weight>
  </element>
  <element>
    <symbol>C</symbol>
    <number>6</number>
    <weight>12</weight>
  </element>
  <element>
    <symbol>O</symbol>
    <number>8</number>
    <weight>16</weight>
  </element>
</elements>
```

Обозначение элемента	Номер	Атомная масса
H	1	1
C	6	12
O	8	16

```
CSV
H,1,1;
C,6,12;
O,8,16;
```

```
# JSON
{"elements":
  [{"symbol": "H",
    "number": 1,
    "weight": 1},
  {"symbol": "C",
    "number": 6,
    "weight": 12},
  {"symbol": "O",
    "number": 8,
    "weight": 16}]}
```

```
# YAML
elements:
  - symbol: H
    number: 1
    weight: 1
  - symbol: C
    number: 6
    weight: 12
  - symbol: O
    number: 8
    weight: 16
```

JSON



JavaScript Object Notation

- Широко используется отдельно от JavaScript



Служит для представления структурированных данных

- Синтаксис проще и компактнее чем XML

```
{  
  "firstName": "Иван",  
  "lastName": "Иванов",  
  "address": {  
    "streetAddress": "Московское ш., 101, кв.101",  
    "city": "Ленинград",  
    "postalCode": "101101"  
  },  
  "phoneNumbers": ["812 123-1234", "916 123-4567"]  
}
```



JSON Schema – основанный на XML Schema подход к описания структур данных JSON (использует синтаксис JSON)

- На практике используется редко

YAML

- 🌀 Yet Another Markup Language или YAML Ain't Markup Language
- 🌀 Ориентирован на простоту восприятия человеком
 - 🌀 Вложенность обозначается отступами
 - 🌀 Массивы обозначаются дефисами
 - 🌀 Строки могут записываться без кавычек
 - Кавычки нужны для экранирования специальных символов

```
firstName: "Иван"
lastName: "Иванов"
address:
  streetAddress: "Московское ш., 101, кв.101"
  city: "Ленинград"
  postalCode: "101101"
phoneNumbers:
  - "812 123-1234"
  - "916 123-4567"
```

Что такое OpenAPI?

- 🌐 OpenAPI — это формализованная спецификация и экосистема инструментов, которая предоставляет интерфейс описания HTTP API. OpenAPI не зависит от языков программирования и позволяет генерировать исходный код для библиотек клиентских приложений, текстовую документацию для пользователей, варианты тестирования и другие элементы на основе спроектированной спецификации для некоторого сервиса

Преимущества OpenAPI (1/2)

Документация

- ⦿ Когда у вас есть HTTP API, важно объяснить другим разработчикам, как им пользоваться. OpenAPI позволяет создавать документацию автоматически. Это значит, что вы можете легко показать другим людям, какие данные нужно передать вашему приложению, и что оно вернёт в ответ.

Совместимость

- ⦿ Разные приложения часто используют разные языки программирования и технологии. OpenAPI делает так, чтобы все разработчики могли понимать друг друга независимо от того, какой язык программирования они используют. Он описывает API на простом и понятном языке, который читается всеми одинаково.

Преимущества OpenAPI (2/2)

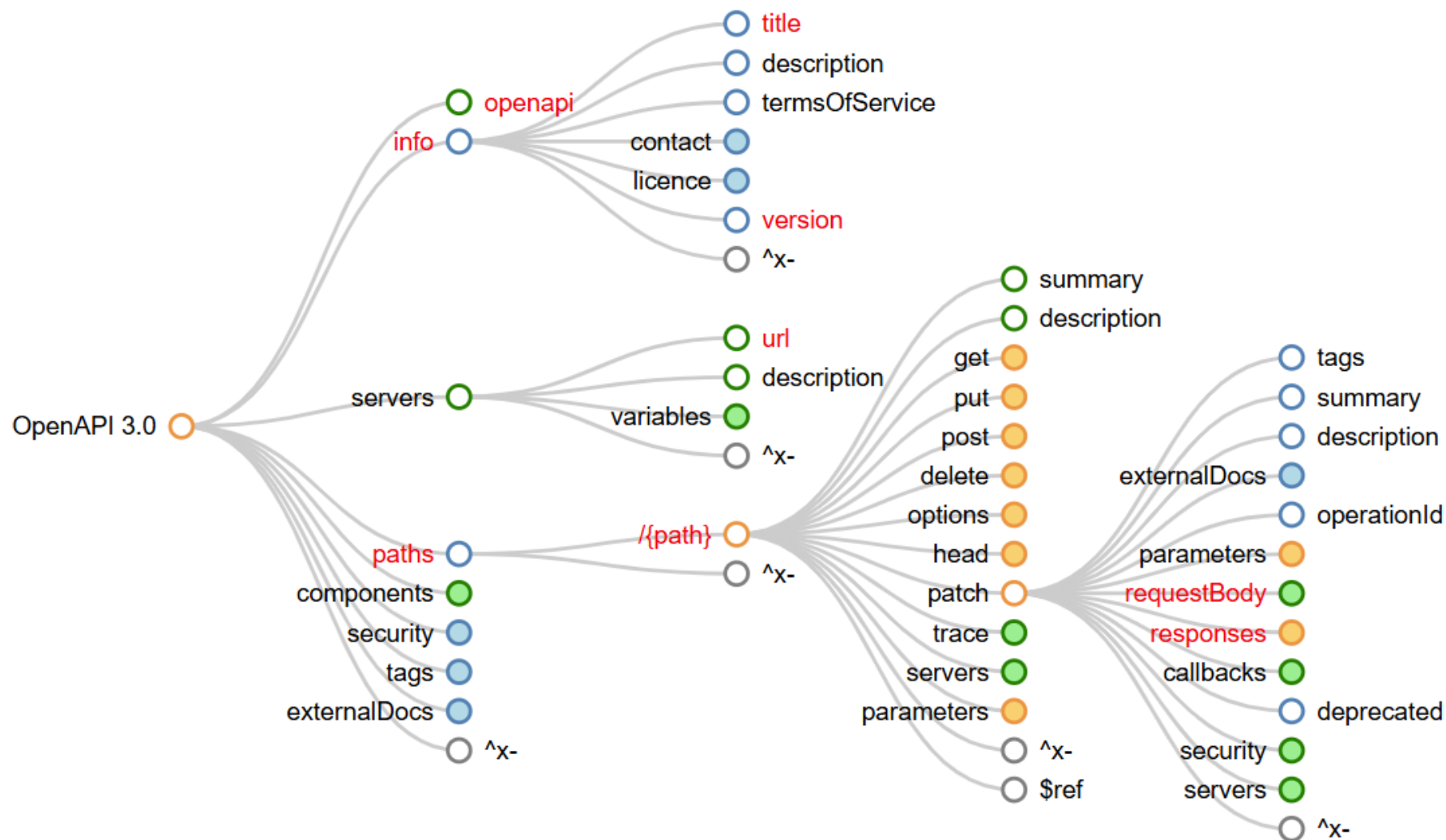
Автоматизация

- ⦿ С помощью OpenAPI можно генерировать код для работы с вашим API. Например, если вы создаёте API для своего сервиса, другой разработчик может использовать этот документ, чтобы автоматически создать программу, которая будет обращаться к вашему сервису. Это экономит много времени и снижает вероятность ошибок.

Тестирование

- ⦿ OpenAPI также упрощает тестирование вашего HTTP API. Вы можете использовать инструменты, которые проверяют, соответствует ли ваше реальное поведение заявленному в документации. Это помогает убедиться, что всё работает правильно.

Карта спецификации



Элементы файла спецификации

- 📶 **Путь (path)** — указывает, куда отправляется запрос. Например, ``/users`` может означать путь к списку всех пользователей
- 📶 **Методы HTTP-запросов (methods)** — определяют типы запросов, такие как GET (чтение данных), POST (создание новых данных), PUT (обновление данных) и DELETE (удаление данных)
- 📶 **Параметры (parameters)** — указывают, какие дополнительные данные должны быть переданы в запросе. Например, параметр ``id`` может использоваться для идентификации конкретного пользователя
- 📶 **Ответы (responses)** — описывают возможные результаты выполнения запроса. Например, успешный ответ (код 200) или ошибка (код 404 — страница не найдена)
- 📶 **Схемы данных (schemas)** — показывают, какую структуру имеют данные, передаваемые в запросах и ответах. Например, схема может описывать, что объект пользователя включает имя, возраст и адрес электронной почты

Пример спецификации в формате YAML (1/6)

```
openapi: 3.0.0
info:
  title: Blog API
  description: API для управления статьями в блоге
  version: 1.0.0
servers:
  - url: https://api.example.com/v1
    description: Production server
```

Пример спецификации в формате YAML (2/6)

```
paths:
  /articles:
    get:
      summary: Список всех статей
      responses:
        '200':
          description: Успешный запрос
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Article'
              example:
                - id: 1
                  title: "Первая статья"
                  author: "Иван Иванов"
                - id: 2
                  title: "Вторая статья"
                  author: "Петр Петров"
```

Пример спецификации в формате YAML (3/6)

```
post:
  summary: Создать новую статью
  requestBody:
    required: true
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ArticleCreate'
        example:
          title: "Новая статья"
          content: "Текст статьи..."
          author: "Аноним"
  responses:
    '201':
      description: Статья создана
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Article'
```

Пример спецификации в формате YAML (4/6)

```
/articles/{id}:  
  get:  
    summary: Получить статью по ID  
    parameters:  
      - in: path  
        name: id  
        required: true  
        schema:  
          type: integer  
        description: ID статьи  
    responses:  
      '200':  
        description: Успешный запрос  
        content:  
          application/json:  
            schema:  
              $ref: '#/components/schemas/Article'  
      '404':  
        description: Статья не найдена
```

Пример спецификации в формате YAML (5/6)

```
components:
  schemas:
    Article:
      type: object
      properties:
        id:
          type: integer
          example: 1
        title:
          type: string
          example: "Заголовок статьи"
        content:
          type: string
          example: "Текст статьи..."
        author:
          type: string
          example: "Автор статьи"
      required:
        - id
        - title
        - author
```


Пример спецификации в формате YAML (6/6)

```
ArticleCreate:
  type: object
  properties:
    title:
      type: string
    content:
      type: string
    author:
      type: string
  required:
    - title
    - content
```

Итоги

В данной лекции были рассмотрены следующие темы:

-  Программные интерфейсы приложений
-  Создание открытых экосистем ПО
-  RESTful API и средства их описания
-  Спецификация OpenAPI